DESIGN PATTERNS · 专题精讲

装饰模式
实战精讲

从原书精髓到订单价格重构
把"包装而非继承"的设计哲学落到可运行代码

本专题围绕《深入设计模式》中最具实战价值的结构型模式——装饰模式（Decorator）展开。先拆解原书核心：通知库子类爆炸问题、聚合优于继承的原则、五角色结构与数据加密压缩经典实例；再进入真实业务重构：电商订单价格计算的"上帝方法 → 装饰栈"完整演进，配可运行 Python 代码、装饰栈执行流可视化、与适配器/代理/责任链/组合/策略五种相邻模式的精确辨析，最后给出常见误用与反模式。

TOPIC

基于原著：**亚历山大·什韦茨**
《Dive Into Design Patterns》
装饰模式章节 · p.182-198

实战专题
共 8 页 · 含 Python 可运行代码

CONTENTS

· 本专题目录

从原书理论到订单重构实战，一条龙贯通装饰模式的"为什么 · 是什么 · 怎么用 · 别误用"。

第一部分　定义与本质 包装代替继承 / 通知库子类爆炸问题 P.03

第二部分　结构与经典实例 五角色 / 数据加密压缩 Python 代码 P.04

第三部分　与相邻模式辨析 适配器 / 代理 / 责任链 / 组合 / 策略 P.05

第四部分　订单重构：上帝方法的问题 坏味道代码诊断 / 子类爆炸的诱惑 P.06

第五部分　订单重构：装饰栈方案 完整 Python 实现 / 执行流可视化 P.07

第六部分　常见误用与小结 四类反模式 / 何时用何时不用 P.08

5
装饰模式结构角色

5
种相邻模式辨析

3
个订单价格计算版本

4
类常见误用反模式

阅读建议

若你不熟悉装饰模式，按一→二→四→五的顺序读（先懂原理再看实战）；若你已熟悉原理只想看重构，直接跳到第四部分。第三部分的模式辨析是面试与代码评审的高频考点，建议精读。

PART 01 · DEFINITION & ESSENCE

一 定义与本质

亦称：装饰者/装饰器模式 | Wrapper · Decorator | 类别：结构型

装饰是一种**结构型设计模式**，允许你通过将对象放入包含行为的特殊封装对象中来为原对象绑定新的行为。一句话本质：**用"包装"代替"继承"**——给对象套一层外壳，外壳既能转发请求，又能在转发前后加点料，可多层叠加，像穿衣服一样。

问题：通知库的子类爆炸（原书 p.182-183）

原书用一个非常典型的例子开篇。你的通知库最初只发邮件：

✗ 方案 A：继承

用户要短信、微信、QQ、钉钉……每种渠道做一个子类；用户还要"短信+微信""短信+QQ""全要"——**N 个渠道 → 2^n-1 个子类**。再加一个渠道，子类翻倍。原书称之为"打破吉尼斯纪录"。

✓ 方案 B：装饰

每个渠道做成一个**装饰器**，运行时按需套层。N 个渠道只需 **N 个装饰类 + 1 个基础类**，任意组合。

为什么不能继承？原书的三大理由

继承的问题	装饰怎么解决
静态	运行时无法改对象行为，只能整个换对象 → 装饰是**动态**的，运行时套/脱
单继承	多数语言不能同时继承多个父类 → 装饰基于**组合**，可包多个
子类爆炸	每种组合都要一个子类 → 装饰**线性增长**，自由组合

原书关键原则

「聚合（或组合）是许多设计模式背后的关键原则」——装饰、策略、状态、代理都建立在此之上。

聚合：A 包含 B，B 可独立于 A 存在（装饰用聚合）。组合：A 由 B 构成，A 管理 B 的生命周期（B 不能独立）。装饰用的是**聚合**。

真实世界类比：穿衣服（原书 p.188）



觉得冷 → 穿毛衣；还冷 → 套夹克；下雨 → 再穿雨衣。所有衣物都"扩展"了你的基本行为，但**它们不是你的一部分**，不需要时随时脱掉。精妙之处：衣服是**可拆卸**的（运行时增删），人是**唯一**的（具体部件）。继承做不到——你没法"临时长出一件毛衣"。

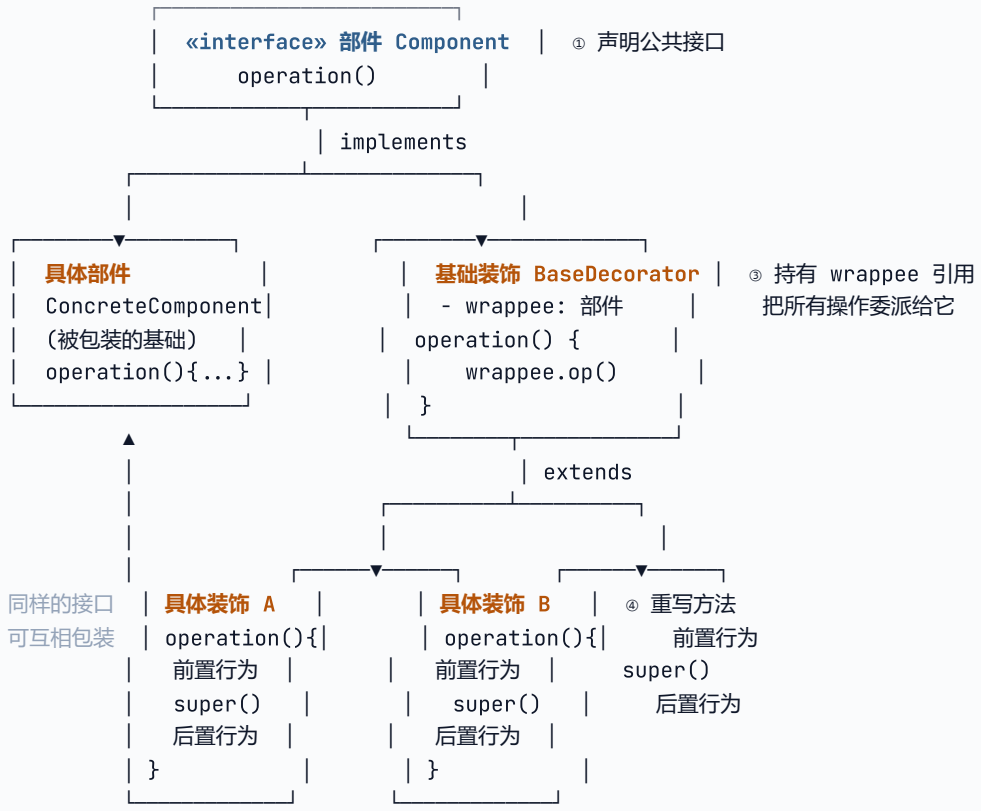
CORE INSIGHT

装饰模式的全部魔力来自一个简单事实：**装饰器实现了和被包装对象相同的接口**。所以从客户端看，套不套装饰完全一样——这正是能"层层套"的前提。每一层都以为自己在调用一个普通部件，却不知道对方其实也是个装饰器。

PART 02 · STRUCTURE & CLASSIC EXAMPLE

二 结构与经典实例

五角色结构（原书 p.189-190）



经典实例：数据加密 + 压缩（原书 p.190-194，转 Python）

原书最经典的实战。基础对象是文件读写，装饰器负责加密、压缩。我转成**可直接运行的 Python**：

decorator_data_source.py · 可运行 PYTHON

```python
from abc import ABC, abstractmethod
import base64, zlib

# ① 部件接口：声明读写数据的方法
class DataSource(ABC):
    @abstractmethod
    def write_data(self, data: str) -> None: ...
    @abstractmethod
    def read_data(self) -> str: ...

# ② 具体部件：读写文件（被包装的基础对象）
class FileDataSource(DataSource):
    def __init__(self, filename): self.filename = filename
    def write_data(self, data):
        with open(self.filename, 'w', encoding='utf-8') as f: f.write(data)
    def read_data(self):
        with open(self.filename, 'r', encoding='utf-8') as f: return f.read()

# ③ 基础装饰：持有引用，默认原样委派
class DataSourceDecorator(DataSource):
    def __init__(self, source: DataSource): self.wrappee = source
    def write_data(self, data):  self.wrappee.write_data(data)
    def read_data(self):         return self.wrappee.read_data()

# ④ 具体装饰 A：加密。写前加密，读后解密
class EncryptionDecorator(DataSourceDecorator):
    def write_data(self, data):
        encrypted = base64.b64encode(data.encode()).decode()  # 前置
        super().write_data(encrypted)                          # 委派
    def read_data(self):
        return base64.b64decode(super().read_data()).decode() # 后置

# ④ 具体装饰 B：压缩。写前压缩，读后解压
class CompressionDecorator(DataSourceDecorator):
    def write_data(self, data):
        compressed = zlib.compress(data.encode()).hex()        # 前置
        super().write_data(compressed)
    def read_data(self):
        return zlib.decompress(bytes.fromhex(super().read_data())).decode()

# ⑤ 客户端：运行时按需组装装饰栈
if __name__ == "__main__":
    salary = "张三,10000;李四,15000"
    # 情况 3：压缩 + 加密（套的顺序很关键！）
    source = EncryptionDecorator(            # 最外层→最后写、最先读
                CompressionDecorator(        # 中间层
                    FileDataSource("salary.dat")))  # 最内层→真正写文件
    source.write_data(salary)
    print(source.read_data())  # → "张三,10000;李四,15000"（完美还原）
```

> ⚠ **装饰顺序很重要（原书 p.193）**
>
> **写入**：加密 → 压缩 → 写文件（外层往内层流）。**读取**：读文件 → 解压 → 解密（内层往外层流，对称还原）。若写成 Compression > Encryption，语义就变了：压缩后的密文 ≠ 加密后的压缩数据。**装饰栈顺序 = 行为执行顺序**。

适用场景（原书 p.194-195）

- **运行时动态增减行为**——不改代码就能加功能，多装饰组合使用
- **继承不可行时**——类被 final 标记、或子类组合爆炸，用包装替代继承
- **单一职责拆分**——把"做太多事的大类"拆成基础行为 + 多个可选装饰层

PART 03 • COMPARING WITH NEIGHBORING PATTERNS

三 与相邻模式辨析

这组辨析是**面试和代码评审的高频考点**。原书 p.196-197 给出了非常清晰的判别标准——核心是看「**接口是否改变**」和「**意图是什么**」。

金句：装饰 vs 适配器 vs 代理（"结构三兄弟"）

> 原书原文
> 「**适配器提供不同的接口，代理提供相同的接口，装饰提供加强的接口。**」

模式	接口	意图	判别要点
适配器 Adapter	改变（A→B）	让不兼容对象协作	Type-C 转 USB
代理 Proxy	相同	**控制访问**（懒加载/权限/远程）	明星经纪人
装饰 Decorator	相同	**增强行为**（前后加料）	礼物包装

关键区别：**代理通常自行管理服务对象的生命周期，装饰的生成总是由客户端控制**。

装饰 vs 责任链

相似

类结构几乎一样：都靠**递归组合**把操作传递给一系列对象。

不同

责任链：每个处理者可独立操作，可随时**中断**请求传递。**装饰**：不能中断传递，目的是**叠加增强**。

装饰 vs 组合

相似

都依赖递归组合组织对象，结构图相似。

不同

组合：树形结构，**多个子节点结果求和**（部门含子部门）。
装饰：链式，**只有一个子组件**，目的是加额外职责。两者可合作：装饰可扩展组合树中特定对象。

装饰 vs 策略（原书最有名的对比金句）

> 原书金句
> 「**装饰让你改变对象的外表，策略让你改变对象的本质。**」

维度	装饰 Decorator	策略 Strategy
对象核心	核心对象**不变**，行为层层包裹	换**内核**，整个算法被替换
关系	装饰器与被包装对象**同接口**	上下文**持有一个**策略引用
可叠加	可套**多层**	通常**一次一个**（可切换）
典型场景	加日志/缓存/加密（横切）	换排序/支付/路线算法

DECISION RULE

判断用装饰还是策略，问自己一个问题：「我要**加层功能**，还是要**换个算法**？」加层 → 装饰；换算法 → 策略。两者结构都基于组合，但意图完全不同。

现实世界中的装饰模式

> 你可能天天在用

Java I/O：三层套的流；**Python @decorator**：@log @cache @retry 语言级支持；**Spring Filter / Express 中间件**：编码/鉴权/日志层层包装请求；**奶茶点单**：基础奶茶 +珍珠 +椰果 +奶盖，价格累加。

Java I/O 是装饰模式最经典的"教科书案例"

初学者对 `new BufferedReader(new InputStreamReader(new FileInputStream("a.txt")))` 套三层感到困惑——但这正是装饰模式的优雅之处：每层独立增强，可自由组合。

Java I/O 装饰栈解析

```
// 最内层：具体部件，真正读文件字节
FileInputStream fis = new FileInputStream("a.txt");

// 第一层装饰：把字节流转成字符流（编码处理）
InputStreamReader isr = new InputStreamReader(fis, "UTF-8");

// 第二层装饰：加缓冲区，减少磁盘 IO（性能增强）
BufferedReader br = new BufferedReader(isr);

// 读取时：br.read() → 缓冲判断 → isr.read() → fis.read() → 磁盘
// 每层各司其职：fis 读字节 / isr 转字符 / br 加缓冲，互不耦合
String line = br.readLine();
```

理解了装饰模式，Java I/O 那些看似繁琐的嵌套就变成了**可组合的设计之美**。想加解密？套一层 CipherInputStream；想加压缩？套一层 GZIPInputStream——完全不用改原有代码。

PART 04 · REFACTORING · THE PROBLEM

四 订单重构：上帝方法的问题

电商订单的最终价格由多层规则叠加：**商品原价 → 会员折扣 → 优惠券 → 满减 → 运费**。需求还在不断加（新人券、跨店满减、平台补贴…）。我们看"坏味道"代码如何诊断。

坏味道 v1：上帝方法（最常见的起点）

绝大多数项目的初版长这样——一个方法里 if-else 堆所有规则：

✗ order_bad.py · 上帝方法

```
class OrderService:
    def calculate_total(self, order):
        # 1. 商品原价
        total = sum(item.price * item.qty for item in order.items)
        # 2. 会员折扣
        if order.user.is_vip:    total *= 0.9
        elif order.user.level == 'gold': total *= 0.95
        # 3. 优惠券
        if order.coupon:
            if order.coupon.type == 'fixed':   total -= order.coupon.value
            elif order.coupon.type == 'percent': total *= (1 - order.coupon.value/100)
        # 4. 满减
        if total >= 300: total -= 30
        # 5. 运费
        if total < 99:                total += 12
        elif order.user.is_remote:    total += 25
        return max(total, 0)
```

问题诊断：违反了哪些原则？

问题	具体表现
单一职责 + 开闭违反	一个方法干 5 件事；加新规则（如"新人券"）必须修改此方法
测试灾难 + 难复用	5 规则 × 多组合 = 笛卡尔积测试用例；促销页只想算券后价却被迫算运费
顺序硬编码	满减在券后、运费在最后——改顺序要动业务代码

坏味道 v2：子类爆炸（继承的诱惑）

有人会想："拆成子类不就好了？" 看看会怎样：

✗ order_bad2.py · 继承地狱

```
class BaseCalculator:
    def calculate(self, order): return sum(i.price*i.qty for i in order.items)

class VIPDiscount(BaseCalculator):
    def calculate(self, order): return super().calculate(order) * 0.9

class FixedCoupon(BaseCalculator):
    def calculate(self, order): return super().calculate(order) - order.coupon.value

# 💥 想要 VIP + 满减券 + 满减 + 运费? 只能多继承
class VIPFixedCouponFullReductionShipping(VIPDiscount, FixedCoupon, ...): ...
```

这正是原书 p.183 说的**"子类组合数量爆炸"**——5 个规则理论上有 2^5-1=31 种组合子类。多继承受限、菱形继承复杂。

THE INSIGHT

根源在于**用"继承"思维理解"组合"**。订单价格不是"一个类的多种变体"，而是"一组规则的**动态叠加**"——这正是装饰模式要解决的。

测试困境：笛卡尔积爆炸

看似简单的 5 个规则，要测全组合需要多少用例？以"会员折扣(3态:无/VIP/金) × 优惠券(3态:无/满减/百分比) × 满减(2态) × 运费(3态)"为例：

> **测试用例爆炸**
>
> 3 × 3 × 2 × 3 = **54 种组合**，每种都要构造完整 Order 对象、断言最终价格。实际项目中规则往往 8-10 个，组合数轻松破千。更痛的是：**改一个规则，所有用例都得重跑**，因为它们共享同一个方法。

装饰模式重构后，每个装饰器**独立测试**：VIPDiscountDecorator 只测"VIP 打 9 折"一个分支，不关心券和运费。54 个用例变成 5 个装饰器各 3-5 个用例，**维护成本骤降**。

> **下一部分**
>
> **问题诊断清楚了：上帝方法违反单一职责与开闭原则，子类方案会爆炸，测试维护成本失控。第五部分将用装饰模式逐步重构——定义部件接口、拆分每个规则为装饰器、用工厂动态组装，并可视化整个装饰栈的执行流。**

PART 05 · REFACTORING · THE SOLUTION

五 订单重构：装饰栈方案

按原书 p.195 六步实现法：先抽象"价格计算"接口，再让每个规则变装饰器。

order_decorator.py · 完整可运行方案 PYTHON

```python
from abc import ABC, abstractmethod

# ① 部件接口 (Component)
class PriceCalculator(ABC):
    @abstractmethod
    def calculate(self, order) -> float: ...

# ② 具体部件：商品原价
class BasePriceCalculator(PriceCalculator):
    def calculate(self, order):
        return sum(i.unit_price * i.quantity for i in order.items)

# ③ 基础装饰器
class PriceDecorator(PriceCalculator):
    def __init__(self, wrapped: PriceCalculator): self._wrapped = wrapped
    def calculate(self, order): return self._wrapped.calculate(order)

# ④ 具体装饰：会员折扣 / 优惠券 / 满减 / 运费
class VIPDiscountDecorator(PriceDecorator):
    def calculate(self, order):
        base = super().calculate(order)
        return base * 0.9 if order.user.is_vip else base

class CouponDecorator(PriceDecorator):
    def calculate(self, order):
        base = super().calculate(order)
        if not order.coupon: return base
        if order.coupon.type == 'fixed':   return base - order.coupon.value
        if order.coupon.type == 'percent': return base * (1 - order.coupon.value/100)
        return base

class FullReductionDecorator(PriceDecorator):
    def calculate(self, order):
        base = super().calculate(order)
        return base - 30 if base >= 300 else base

class ShippingDecorator(PriceDecorator):
    def calculate(self, order):
        base = super().calculate(order)
        ship = 12 if base < 99 else 0
        if order.user.is_remote: ship += 25
        return base + ship

# ⑤ 工厂：根据业务上下文动态组装装饰栈
def build_calculator(order, ctx) -> PriceCalculator:
    calc = BasePriceCalculator()                        # 最内层
    if ctx.get('enable_member'):
        if order.user.is_vip:           calc = VIPDiscountDecorator(calc)
    if order.coupon:                    calc = CouponDecorator(calc)
    if ctx.get('enable_full_reduction'): calc = FullReductionDecorator(calc)
    if ctx.get('include_shipping'):     calc = ShippingDecorator(calc)
    return calc
```

装饰栈执行流可视化

以"VIP + 满减券 + 满300 + 偏远地区"为例。向下是递归调用，向上是逐层返回加料：

```
ShippingDecorator 最外层 · 最后算（加运费）  base = super() → base < 99 ? +12 ; remote ? +25 → return base + shipping
FullReductionDecorator 满 300 减 30  base = super() → base ≥ 300 ? -30 → return base
CouponDecorator 扣优惠券  base = super() → coupon.type=='fixed' ? -val → return base
VIPDiscountDecorator 9 折  base = super() → is_vip ? *0.9 → return base * 0.9
BasePriceCalculator 最内层 · 最先算（商品原价）  return sum(item.unit_price * item.quantity) ← 真正的起点
```

前后对比：装饰模式带来了什么

维度	v1 上帝方法	v3 装饰模式
新增规则	改 calculate_total，加 if 分支	新建装饰类，**零修改现有代码**（开闭）
单元测试	必须模拟全栈组合	每个装饰器**独立测试**（单职责）
规则复用	不行，方法不可拆	促销页可只组装"原价+券"，不算运费
顺序调整	改业务代码里的 if 顺序	改工厂组装顺序，业务代码不动

PART 06 • ANTI-PATTERNS & CONCLUSION

六 常见误用与小结

四类常见误用（实战经验）

✗ 误用 1：顺序敏感却没文档化

谁也不知道该先套 Logging 还是 Auth。**对策**：用 Builder 或工厂封装"推荐组合"，别让客户端自己拼。

✗ 误用 2：破坏接口契约

如 ReadOnlyCollection 装饰器对 add() 抛异常——违反里氏替换。**对策**：装饰器应保持接口契约，只增强不削弱。

✗ 误用 3：层数过多

5 层以上栈帧深、报错难定位。**对策**：控制在 3 层内，超过考虑重新设计。

✗ 误用 4：当万能加料机

多个装饰器间有状态共享/强耦合逻辑时很痛苦。**对策**：改用责任链或中介者。

优缺点（原书 p.196）

✓ 优点

① 无需新建子类即可扩展行为；② 运行时**添加/删除**功能；③ 多装饰组合，行为可叠加；④ 单一职责：大类拆成小类。

✗ 缺点

① 从栈中**删除特定装饰器**较困难；② 实现不受栈顺序影响的装饰较难；③ 多层初始化配置代码**看着丑**。

决策清单：什么时候用装饰模式?

用 ✓	不用 ✗
需要运行时动态增减功能	行为变化很少、编译期就能确定 → 直接继承更简单
继承不可行（final 类）或子类爆炸	装饰器间有复杂依赖/顺序耦合 → 责任链或中介者
想把"核心职责"和"可选增强"解耦	只需"换个算法"而非"加层功能" → 策略模式

FINAL TAKEAWAY

装饰模式的本质是**「组合优于继承」**这一原则的极致体现。它不是让你写更多的类，而是让你把变化**隔离**——每个规则独立演化，互不干扰，还能在运行时自由组合。当你下次再面对一个 if-else 堆满规则的方法时，先问一句：「这些规则，能不能拆成可插拔的装饰器？」

EPILOGUE

组合，而非继承

「聚合（或组合）是许多设计模式背后的关键原则。」
——《深入设计模式》

装饰模式只是组合思维的一个应用。当你真正内化了"**运行时把行为组合起来**"这件事，你会发现策略、状态、代理、责任链、命令……它们都是同一思想在不同问题上的投影。

NEXT · 深入
代理模式：装饰的"控制访问"孪生兄弟

NEXT · 实战
HTTP 客户端加重试/日志/熔断装饰栈